
Feature Engineering for Classification-Based Merge Conflict Resolution

Ángel Oroz Azcarate¹

1. Introduction

Software merging is an essential yet error-prone activity in modern collaborative development. When multiple developers modify the same sections of a codebase concurrently, version control systems such as Git produce *merge conflicts* that require manual resolution. Empirical studies indicate that 10–20% of merge attempts result in conflicts (Boll et al., 2024), and resolving them can take hours to days of developer effort (Elias et al., 2023).

A promising direction for automating conflict resolution exploits the empirical observation that the vast majority of conflicts found in practice can be resolved using a small fixed set of *conflict resolution patterns*, combinations of the OURS, THEIRS, and BASE versions of a conflicting chunk (Ghiotto et al., 2020). This reduces the problem to a supervised multi-class classification task: given a feature representation of a conflicting chunk, predict which resolution pattern a developer would apply.

The most comprehensive study in this line of research, MESTRE (Elias et al., 2023), trained and evaluated several classical classifiers on 20 open-source Java projects, reporting a best accuracy of approximately 81% with a Random Forest. A subsequent replication study (Authors, 2026) identified a critical experimental pitfall in MESTRE: *split-related data leakage*, occurring when conflicting chunks from the same merge commit are distributed across both training and test sets. Since developers tend to apply the same resolution pattern to all chunks of a given merge, this leakage artificially inflates measured performance. Once corrected by enforcing merge-level grouping, accuracy drops to 0.66 on the same dataset, and to 0.65–0.68 under cross-project and cross-language generalization settings.

This project addresses a dimension left largely unexplored by prior work: the role of feature representation in classification-based merge conflict resolution. The observed performance ceiling raises a key question: whether

this limitation is due to the learning algorithms themselves, or to the way merge conflicts are represented through meta-data features. In particular, the presence of redundancy and multicollinearity in the feature space suggests that feature selection and engineering may play a critical role. We therefore investigate whether improving the feature space can lead to better predictive performance and more consistent model behavior. To assess generalizability, we evaluate our approach on two datasets: the reproduction-oriented Java dataset from (Authors, 2026) and a large-scale multi-language dataset covering 10 programming languages.

Contributions. This work investigates the role of feature representation in classification-based merge conflict resolution under a leakage-free evaluation setting. Our contributions are as follows:

1. We provide a systematic analysis of redundancy and multicollinearity in the feature space used for merge conflict classification.
2. We conduct a controlled empirical study of feature selection and dimensionality reduction techniques across multiple classifiers and feature subset sizes.
3. We analyze the relationship between feature selection, model behavior, and predictive performance, identifying which types of features are consistently informative.
4. We evaluate generalizability across programming languages, showing that the main findings hold beyond Java under cross-project conditions.

2. Background and Related Work

2.1. Merge Conflict Resolution as a Classification Problem

In Git’s three-way merge model, a conflict arises when two branches (OURS and THEIRS) have made incompatible changes to the same region of a file relative to their common ancestor (BASE). The conflict is externalized as a *conflicting chunk* in the tentative merge result, which the developer must resolve manually.

¹Student of the Swiss Joint Master in Computer Science, University of Bern, Switzerland. Correspondence to: Ángel Oroz Azcarate <angel.oroazcarate@students.unibe.ch>.

Ghiotto et al. (Ghiotto et al., 2020) conducted a large-scale empirical study on 2,731 Java projects and established that the vast majority of chunk resolutions follow one of a small number of canonical patterns: selecting exactly OURS, exactly THEIRS, keeping BASE, deleting the chunk entirely (EMPTY), or combining two or three of these variants. Resolutions that do not conform to any pattern are classified as *non-canonical* and require custom manual edits.

Building on this taxonomy, MESTRE (Elias et al., 2023) extracted 43 features per conflicting chunk (covering authorship, change magnitude, temporal characteristics, semantic signals from commit messages, contextual complexity, and chunk size/location) and trained classical classifiers to predict resolution patterns. Their best model, a Random Forest, achieved $\sim 81\%$ accuracy in an intra-project setting. However, as identified by (Authors, 2026), this result was inflated by split-related data leakage at an average rate of 96%. After controlling for leakage via merge-level grouping, the same experimental setup yields only $\sim 66\%$ accuracy, and realistic cross-project and cross-language settings further reduce performance to 0.65–0.68. The authors conclude that classical feature-based ML approaches may be approaching their practical performance ceiling for this problem, and that fundamentally different strategies, such as large language models operating on the actual code content, may be required for meaningful further progress.

2.2. Feature Selection Methods

Feature selection aims to identify a subset of relevant features that preserves or improves predictive performance while reducing overfitting, lowering computational cost, and enhancing model interpretability. Methods are typically grouped into three categories (Guyon & Elisseeff, 2003).

Filter methods evaluate features independently of any learning algorithm, using statistical criteria. Common approaches include *Chi-square* (χ^2), which measures dependence between categorical variables, and *Mutual Information* (MI), which captures both linear and non-linear dependencies between features and labels.

Wrapper methods select features based on the performance of a predictive model. These include *Recursive Feature Elimination* (RFE), which iteratively removes the least important features, and *Sequential Feature Selection* (SFS/SBS), which adds or removes features greedily. While capable of modeling feature interactions, these methods are computationally expensive.

Embedded methods integrate feature selection into the model training process. Linear models with L1 regularization, such as *LASSO*, can drive coefficients of irrelevant features to zero. *ElasticNet* extends this approach to better

handle correlated features. Tree-based models provide alternative importance measures, such as mean decrease in impurity in Random Forests.

Dimensionality reduction methods, such as *Principal Component Analysis* (PCA), transform the original feature space into a lower-dimensional representation. Unlike feature selection, these methods construct new features as combinations of the original ones, often improving compactness at the cost of interpretability.

2.3. Classification Models

Random Forest (RF) is an ensemble of decision trees trained on bootstrap samples with random feature subsampling at each split (Breiman, 2001). Its inherent randomization makes it robust to feature redundancy (correlated features are naturally distributed across trees) and to overfitting. It serves as the state-of-the-art baseline for this problem (Elias et al., 2023; Authors, 2026).

Logistic Regression (LR) is a linear classifier that estimates class probabilities via a softmax function of a weighted feature sum. LR is highly sensitive to multicollinearity: when two features are strongly correlated, their coefficients become unstable and the optimizer may fail to converge, degrading predictive performance. This sensitivity makes LR an ideal model for studying the impact of feature selection in the presence of redundant features.

Decision Tree (DT) partitions the feature space by recursive binary splits chosen to maximize information gain or Gini impurity reduction. Unlike RF, a single DT lacks the variance reduction provided by ensembling and is therefore sensitive to noisy or redundant features, which can lead to overfitting and instability across folds. Feature selection is expected to act as an implicit regularizer for DT.

XGBoost (XGB) implements gradient boosted trees (Chen & Guestrin, 2016), building trees sequentially where each new tree corrects the residual errors of the previous ensemble. XGBoost includes built-in L1 (α) and L2 (λ) regularization on the tree weights, which can suppress irrelevant features during training. Its sequential boosting mechanism allows it to focus on difficult examples (including minority classes) across iterations.

Support Vector Machine (SVM) finds the maximum-margin hyperplane separating classes in a kernel-induced feature space. They can model complex decision functions but scale poorly with dataset size.

K-Nearest Neighbours (KNN) classifies each sample by the majority vote of its k nearest neighbours in feature

space. KNN suffers from the curse of dimensionality: as the number of features grows, Euclidean distances become less discriminative. Additionally, inference requires computing distances to all training points ($O(n \cdot d)$), making it slow for large datasets.

3. Experimental Setup

3.1. Datasets

Java dataset (RQ1). We use the reproduction-oriented dataset from (Authors, 2026), which re-mines the 20 Java projects originally studied by (Elias et al., 2023) from GitHub. After filtering projects with fewer than 1,000 conflicting chunks, 16 projects remain, comprising 141,127 conflicting chunks across 9,748 merge commits. Table 1 summarizes the per-project statistics.

Table 1. Java dataset statistics. Projects ordered by number of chunks.

Project	Merges	Chunks
aosp-mirror/platform	4,500	84,514
apache/accumulo	1,788	14,848
CCI-MIT/XCoLab	333	9,323
zkoss/zk	675	6,694
TeamDev/OpenFaces	176	3,914
eucalyptus/eucalyptus	577	3,153
hibernate/orm	145	2,707
jgralab/jgralab	99	2,537
apache/lucene-solr	285	2,326
wro4j/wro4j	351	2,215
alkacon/opencms-core	43	1,726
hyphane/fred	260	1,425
Unidata/thredds	130	1,391
CloudStack-archive	128	1,223
TelluIoT/ThingML	52	1,130
getrailo/railo	156	1,001
Total	9,748	141,127

Multi-language dataset. To assess whether findings generalize beyond Java, we additionally evaluate on the large-scale multi-language dataset from (Authors, 2026), covering 10 programming languages (C, C++, C#, Go, Java, JavaScript, PHP, Python, Rust, TypeScript) drawn from the top-1,000 GitHub repositories per language by star count. This dataset contains over 4 million conflicting chunks across more than 700 projects, and is evaluated under a cross-project generalization setting: for each language, a project-based 5-fold cross-validation is performed where training and test sets contain *disjoint sets of projects*, mirroring the cross-project evaluation of (Authors, 2026).

Both datasets share the same 43-feature representation. The classification target comprises 6 resolution pattern classes. In the Java dataset, the class distribution is highly imbal-

anced: CANONICAL-OURS (73.9%), NONCANONICAL (13.3%), CANONICAL-THEIRS (9.7%), and three minority classes each below 1%. The multi-language dataset exhibits a similar but slightly less extreme imbalance, as the OURS dominance varies across languages.

Each chunk is represented by 43 features organized into six groups: (1) *Authorship and Collaboration*, (2) *Change Magnitude*, (3) *Temporal Characteristics*, (4) *Semantic Signals* (12 commit-message keyword counts), (5) *Contextual Complexity*, and (6) *Chunk Size and Location*.

3.2. Evaluation Protocol

Java dataset. To prevent split-related data leakage (Authors, 2026), all experiments use **merge-level grouping**: all chunks belonging to the same merge commit are assigned entirely to either the training or the test fold. We implement this via `StratifiedGroupKFold` with 5 folds and `groups = merge_id`, stratified by resolution pattern. Experiments are conducted per project; global performance is the mean across 16 projects.

Multi-language dataset. We use **project-level grouping**: `StratifiedGroupKFold` with `groups = project_id`, so that each fold trains on a subset of projects and tests on entirely unseen projects. This directly replicates the cross-project generalization setting of (Authors, 2026) and ensures that the model never sees any chunk from a test project during training. We run this procedure independently for each of the 10 languages.

Metrics. We report the following metrics:

- **Accuracy**: proportion of correctly classified chunks; directly comparable with (Elias et al., 2023; Authors, 2026).
- **F1-Weighted**: weighted average F1-score, accounting for class imbalance by weighting each class by its support.
- **AUC-OvR**: area under the ROC curve using a one-vs-rest decomposition with weighted averaging; measures discriminative ability independently of the decision threshold.
- **Jaccard Stability**: mean Jaccard index across all pairs of CV folds for the selected feature sets, measuring selection consistency. A value of 1.0 indicates identical feature sets in every fold.

Statistical testing. Statistical significance was assessed using the Wilcoxon signed-rank test. Across most configurations, no statistically significant differences were observed

between the reduced and complete feature sets. This indicates that feature selection generally preserved predictive performance while substantially reducing the number of features required by the models.

3.3. Baseline Replication

We replicate two baselines from (Authors, 2026): the intra-project leakage-controlled setting on the Java dataset (their S3 evaluation), and the cross-project generalization setting on the multi-language dataset (their Table IV, cross-project column).

Java dataset replication. Table 2 compares our per-project results against the reported values. Our replication achieves a mean accuracy of 0.647 and F1-score of 0.621, consistent with the leakage-controlled baseline of (Authors, 2026). Minor per-project discrepancies are expected due to differences in random seeds and dataset reconstruction timing. The global trend confirms the substantial performance drop from the originally reported 0.83 accuracy once split-related leakage is eliminated.

Multi-language dataset replication. Table 3 compares our cross-project RF results per language against the values reported in (Authors, 2026). The relative ordering across languages and the mean accuracy (0.671 vs. 0.65) are consistent with the paper, confirming that our cross-project experimental setup correctly replicates the intended evaluation conditions.

3.4. Feature Selection Methods Employed

Based on the considerations in Section 2.2, we select the following methods for evaluation:

- **Chi-square (chi2):** applied after MinMaxScaling to $[0, 1]$ (required for non-negativity). Evaluated for its interpretability and efficiency as a filter baseline.
- **Mutual Information (MI):** applied directly without scaling, as MI is not distance-based. Selected for its ability to capture non-linear feature-label dependencies.
- **RFE:** uses a Decision Tree as the internal estimator for all models, replacing the actual classifier. This is a deliberate design choice: using RF or XGBoost as the RFE estimator requires hundreds of full model fits per fold, making the experiment computationally intractable. A DT provides comparable importance rankings at a fraction of the cost.
- **RF Importance:** trains a dedicated Random Forest (`n_estimators=100`, `class_weight=balanced`) and selects the

top- k features by mean decrease in impurity. The `class_weight=balanced` compensates for the heavy class imbalance during the importance computation.

- **PCA:** applied after StandardScaling, with `n_components=0.95` to retain 95% of explained variance. Included as a dimensionality reduction baseline to contrast transformation-based vs. selection-based approaches.

Discarded methods. SFS and SBS were evaluated in preliminary experiments and discarded due to their $O(k^2)$ computational cost: a single run with $k = 10$ required over one hour per model. LASSO and ElasticNet (embedded LR-based selection) failed to converge even with 10,000 iterations, as the 43-feature space with severe multicollinearity produces a near-singular design matrix that the LBFGS solver cannot invert reliably.

Feature subset sizes. We evaluate $k \in \{10, 20, 30, 40\}$ for all selection methods. This range covers: aggressive selection ($k=10$, 23% of features), moderate selection ($k=20$, 46%), conservative selection ($k=30$, 70%), and near-complete ($k=40$, 93%).

3.5. Models Employed

We evaluate four classifiers: **Random Forest (RF)** (`n_estimators=400`, `criterion=entropy`, `max_features=0.3`), **Logistic Regression (LR)** (`max_iter=10000`, `class_weight=balanced`), **Decision Tree (DT)** (`class_weight=balanced`), and **XGBoost (XGB)** (`n_estimators=500`, `max_depth=5`, `learning_rate=0.05`, `subsample=0.8`, `colsample_bytree=0.3`, `reg_alpha=0.5`, `min_child_weight=5`).

We exclude Support Vector Machines (SVM) and K-Nearest Neighbours (KNN) from the final experimental setup. SVM is omitted due to its prohibitive training complexity ($O(n^2)$ – $O(n^3)$), which is impractical given the dataset size (141k samples). KNN is excluded due to poor scalability at inference time and its sensitivity to high-dimensional feature spaces, which leads to degraded distance discrimination under the original 43-feature representation.

Among the evaluated models, only Logistic Regression requires feature scaling, which is performed using standardization within each cross-validation fold. All tree-based models (RF, DT, and XGB) are invariant to monotonic feature transformations and therefore do not require scaling. Feature selection is applied prior to scaling within each fold to prevent information leakage.

Table 2. Java dataset: comparison between original results (S3 setting) and our replication. Results in bold highlight projects where deviations are most notable.

Project	Acc (Paper)	Acc (Rep.)	F1 (Paper)	F1 (Rep.)
CCI-MIT/XCoLab	0.64	0.572 ± 0.413	0.61	0.553 ± 0.423
jgralab/jgralab	0.77	0.681 ± 0.118	0.76	0.671 ± 0.172
CloudStack-archive	0.71	0.719 ± 0.106	0.71	0.710 ± 0.114
Unidata/thredds	0.53	0.534 ± 0.098	0.49	0.463 ± 0.114
apache/accumulo	0.69	0.701 ± 0.074	0.65	0.660 ± 0.100
gettrails/railo	0.49	0.484 ± 0.194	0.49	0.473 ± 0.182
zkoss/zk	0.80	0.802 ± 0.043	0.77	0.765 ± 0.063
aosp-mirror/platform	0.92	0.913 ± 0.027	0.92	0.912 ± 0.023
apache/lucene-solr	0.54	0.540 ± 0.079	0.53	0.529 ± 0.072
TeamDev/OpenFaces	0.85	0.845 ± 0.093	0.83	0.814 ± 0.123
hyphanet/fred	0.54	0.560 ± 0.047	0.51	0.528 ± 0.040
wro4j/wro4j	0.64	0.660 ± 0.035	0.63	0.646 ± 0.033
hibernate/orm	0.62	0.618 ± 0.053	0.60	0.584 ± 0.064
eucalyptus/eucalyptus	0.52	0.512 ± 0.056	0.47	0.465 ± 0.038
alkacon/openems-core	0.84	0.777 ± 0.044	0.81	0.750 ± 0.018
TelluIoT/ThingML	0.43	0.439 ± 0.232	0.40	0.418 ± 0.209
Mean	0.66 ± 0.01	0.647 ± 0.107	0.64	0.621 ± 0.112

Table 3. Multi-language dataset: cross-project replication vs. (Auteurs, 2026). Our RF uses tuned hyperparameters; the paper uses a default RF.

Language	Acc (Paper)	Acc (Rep.)
C	.74	0.798 ± 0.107
C#	.66	0.654 ± 0.059
C++	.69	0.677 ± 0.102
Go	.56	0.596 ± 0.067
Java	.71	0.729 ± 0.069
JavaScript	.58	0.574 ± 0.058
PHP	.64	0.653 ± 0.047
Python	.61	0.639 ± 0.090
Rust	.70	0.732 ± 0.088
TypeScript	.64	0.652 ± 0.075
Mean	.65 ± .06	0.671 ± 0.076

4. Feature Space Analysis

Before presenting classification results, we characterize the structure of the 43-feature space to motivate the need for feature selection and interpret its effects.

4.1. Redundancy Analysis: Pearson Correlation

We compute the Pearson correlation matrix over the Java dataset and identify 47 feature pairs with $|r| \geq 0.85$ (51 pairs in the multi-language dataset, reflecting the broader vocabulary of commit messages across languages). The redundancy is concentrated in two groups.

Keyword cluster. The 12 commit-message keyword features (keywords_fix, keywords_bug, keywords_add, etc.) are highly intercorrelated, with

pairwise correlations ranging from 0.85 to 0.99. This is semantically coherent: projects that use one type of keyword heavily tend to use all others proportionally. Despite representing 28% of the feature space (12/43 features), this cluster carries largely redundant information.

Structural size features. chunkRelSizeOURS and chunkRelSizeTHEIRS have a Pearson correlation of exactly -1.0 , as they are complementary by construction: the relative size of OURS plus the relative size of THEIRS sums to a constant. Including both features provides zero additional information and introduces a perfect linear dependency. Similarly, chunkAbsSize and chunkAbsSizeTHEIRS have $r = 0.93$. Figure 1 visualises the full correlation matrix, where both clusters are immediately apparent as dense red blocks.

An additional high-correlation pair appears in the multi-language dataset: changedFilesTHEIRS and concurrentlyChangedFiles ($r = 0.98$), and locAddedTHEIRS and locRemovedTHEIRS ($r = 0.97$). These additional pairs reflect the greater diversity in project activity captured by a multi-language corpus.

4.2. Multicollinearity Analysis: Variance Inflation Factor

Pearson correlation identifies pairwise redundancy but not multicollinearity involving more than two features simultaneously. We compute the Variance Inflation Factor (VIF) for each feature, defined as:

$$\text{VIF}_j = \frac{1}{1 - R_j^2} \quad (1)$$

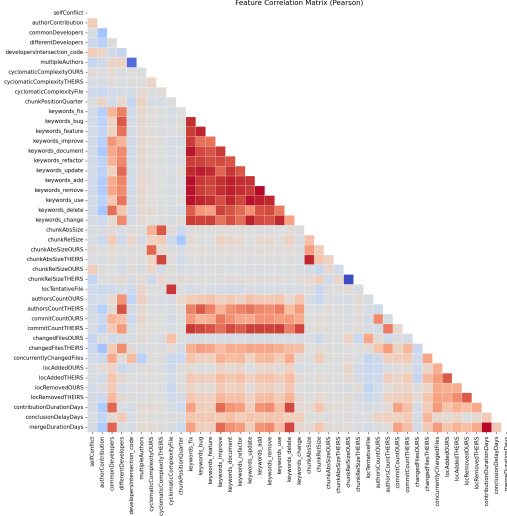


Figure 1. Pearson correlation matrix over the 43-feature space. Two high-correlation clusters are visible: the keyword block (12 commit-message features with pairwise $|r| \geq 0.85$) and the structural size features (lower-right region). The perfect negative correlation between `chunkRelSizeOURS` and `chunkRelSizeTHEIRS` ($r = -1.0$) is the dark blue cell at bottom right.

where R_j^2 is the coefficient of determination from regressing feature j on all other features. $VIF > 10$ indicates severe multicollinearity; $VIF = \infty$ indicates a perfect linear dependency.

The VIF analysis yields nearly identical results across both datasets, as the feature correlations are a property of the feature construction rather than the specific corpus. Table 4 reports the 9 features with $VIF = \infty$ and the 16 features with $10 < VIF < \infty$, totaling 25 out of 43 features (58%) with severe multicollinearity. The keyword features dominate this list, with VIF values ranging from 13 to 210. The infinite VIF features include structural size metrics that are exact linear combinations of one another.

The consequence for LR is particularly severe: with 9 features having $VIF = \infty$, the feature matrix is rank-deficient and the normal equations cannot be solved exactly. This explains the systematic non-convergence warnings observed during LR training and the degraded accuracy relative to tree-based models.

4.3. Feature Selection Stability

We compute the Jaccard stability index for each selection method and feature subset size k , averaged across projects. The Jaccard index between two feature sets A and B is defined as $J(A, B) = |A \cap B| / |A \cup B|$, and we report the mean over all fold pairs within a project, further aver-

Table 4. Features with severe multicollinearity ($VIF > 10$). $VIF = \infty$ denotes exact linear dependencies.

Feature	VIF
<code>commonDevelopers</code>	∞
<code>differentDevelopers</code>	∞
<code>chunkAbsSizeOURS</code>	∞
<code>chunkAbsSize</code>	∞
<code>chunkRelSizeTHEIRS</code>	∞
<code>authorsCountTHEIRS</code>	∞
<code>authorsCountOURS</code>	∞
<code>chunkAbsSizeTHEIRS</code>	∞
<code>chunkRelSizeOURS</code>	∞
<code>commitCountTHEIRS</code>	458.0
<code>keywords_change</code>	210.4
<code>commitCountOURS</code>	137.5
<code>keywords_fix</code>	137.4
<code>keywords_add</code>	134.5
<code>keywords_use</code>	122.9
<code>keywords_remove</code>	102.7
<code>keywords_bug</code>	92.4
<code>mergeDurationDays</code>	77.3
<code>keywords_update</code>	70.8
<code>contributionDurationDays</code>	52.3
<code>keywords_document</code>	31.7
<code>keywords_improve</code>	21.0
<code>keywords_feature</code>	19.0
<code>keywords_delete</code>	16.5
<code>keywords_refactor</code>	13.4

aged across projects. Table 5 reports these values for both datasets.

Table 5. Jaccard stability (mean across projects) per method and k . Top: Java dataset. Bottom: multi-language dataset (cross-project CV).

	$k = 10$	$k = 20$	$k = 30$	$k = 40$
<i>Java dataset</i>				
<code>chi2</code>	0.568	0.671	0.787	0.942
<code>mutual_info</code>	0.757	0.827	0.877	0.940
<code>rfe</code>	0.548	0.649	0.759	0.942
<code>rf_importance</code>	0.801	0.785	0.867	0.972
<i>Multi-language dataset (cross-project)</i>				
<code>chi2</code>	0.827	0.757	0.894	0.905
<code>mutual_info</code>	0.873	0.943	0.917	0.980
<code>rfe</code>	0.714	0.820	0.948	0.980
<code>rf_importance</code>	0.873	0.962	0.974	0.980

Stability is substantially higher on the multi-language dataset across all methods, which is expected: with thousands of diverse projects, fold compositions are more stable and the ranking of informative features converges more reliably. On both datasets, `rf_importance` maintains the highest or tied-highest stability at all values of k .

4.4. Stable Feature Subset

A critical difference emerges between the two datasets when examining which features are stably selected. In the Java dataset, stable features (selected in $\geq 80\%$ of folds by `mutual_info`, `rfe`, and `rf_importance`) are exclusively structural: chunk size, lines of code, and cyclomatic complexity. No keyword feature is stable.

In the multi-language dataset, the stable feature sets expand substantially. With the larger and more diverse training corpus, methods achieve high stability for nearly the full feature set: `rf_importance` and `mutual_info` stabilize 38 out of 43 features, and `rfe` stabilizes 34. Crucially, **keyword features now appear in stable subsets across all methods**, whereas they were consistently excluded in the Java dataset.

This contrast has an important interpretation: in the Java dataset, keyword features add noise because their correlation structure is homogeneous within a monolingual corpus. Across 10 programming languages with distinct development cultures, the keyword profiles of commits vary more substantially, and some keywords genuinely discriminate between project types and their associated resolution patterns. The discriminative value of keyword features is therefore *language-dependent*: they are redundant within a single language but informative in a cross-language setting.

5. Results

This section reports classification results across all models and both datasets. For each model we present baseline performance (all 43 features) followed by the effect of each feature selection method, comparing Java and multi-language results in parallel. Throughout, we highlight a key finding that the current framing underemphasises: *feature selection can reduce the feature space by 50–70% with negligible accuracy loss*, a practically significant result independent of whether the accuracy ceiling itself is raised.

Statistical significance on the Java dataset is assessed via the Wilcoxon signed-rank test ($n=16$ projects, $\alpha=0.05$). Multi-language results are reported descriptively; see Section 3.2 for the rationale.

5.1. Baseline Results

Table 6 reports baseline performance on both datasets.

RF achieves the highest accuracy on both datasets. The multi-language results are uniformly higher for tree-based models (RF: +0.084, XGB: +0.077, DT: +0.011), reflecting the larger and more diverse training data. LR accuracy (Java: 0.390, multi-language: 0.395) remains virtually identical across datasets, confirming that multicollinearity is the binding constraint regardless of corpus size.

Table 6. Baseline performance (all 43 features). Java: merge-level 5-fold CV, mean across 16 projects. Multi-language: project-level 5-fold CV, mean across 10 languages.

Model	Java		Multi-lang	
	Acc	AUC	Acc	AUC
RF	0.647	0.794	0.731	0.884
XGB	0.632	0.781	0.709	0.873
DT	0.572	0.644	0.583	0.670
LR	0.390	0.651	0.395	0.758

5.2. Random Forest

Table 7 reports RF performance under each feature selection method across both datasets.

Table 7. RF accuracy by method and k . Java baseline: 0.647. Multi-language baseline: 0.731. † = significantly worse than Java baseline ($p < 0.05$, Wilcoxon).

Method	k	Java Acc	ML Acc
chi2	10	0.588†	0.702
chi2	20	0.608†	0.717
chi2	25	0.616†	0.720
chi2	30	0.631†	0.722
chi2	35	0.638†	0.727
chi2	40	0.649	0.734
mutual_info	10	0.623†	0.703
mutual_info	20	0.618†	0.731
mutual_info	25	0.627†	0.732
mutual_info	30	0.639	0.729
mutual_info	35	0.646	0.730
mutual_info	40	0.648	0.731
rfe	10	0.634†	0.713
rfe	20	0.638	0.726
rfe	25	0.635†	0.730
rfe	30	0.638†	0.731
rfe	35	0.645	0.731
rfe	40	0.648	0.731
rf_importance	10	0.626†	0.697
rf_importance	20	0.642	0.726
rf_importance	25	0.642	0.728
rf_importance	30	0.647	0.730
rf_importance	35	0.649	0.732
rf_importance	40	0.646	0.732
pca	–	0.616†	0.673

The central finding for RF is not whether feature selection improves accuracy, but how quickly accuracy recovers as k increases from 10 to 20–25. On the Java dataset, `rf_importance` at $k=20$ achieves 0.642 (vs. baseline 0.647, $\Delta = -0.005$, $p=0.30$), a statistically non-significant gap using less than half the original features. On the multi-language dataset, `mutual_info` at $k=20$ already matches

the baseline at 0.731. By $k=25$, all four methods are within 0.002 of the baseline on both datasets.

This confirms that the 43-feature space contains substantial redundancy: a subset of roughly 20–25 features captures essentially all the discriminative signal available to RF. No method produces a statistically significant improvement on Java for RF at any k ; the best result (chi2, $k=40$, 0.649) exceeds the baseline by 0.002 ($p=0.53$). PCA significantly degrades performance on Java (-0.031 , $p<0.001$) and produces the largest drop on multi-language (-0.058), confirming the misalignment between variance and discriminative structure.

5.3. Decision Tree

Table 8. DT accuracy by method and k . Java baseline: 0.572. Multi-language baseline: 0.583. * = significantly better; † = significantly worse ($p<0.05$, Wilcoxon, Java only).

Method	k	Java Acc	ML Acc
chi2	10	0.475 [†]	0.586
chi2	20	0.527 [†]	0.578
chi2	25	0.518 [†]	0.586
chi2	30	0.555 [†]	0.583
chi2	35	0.576	0.589
chi2	40	0.567	0.598
mutual_info	10	0.531 [†]	0.602
mutual_info	20	0.544 [†]	0.601
mutual_info	25	0.557	0.599
mutual_info	30	0.555 [†]	0.588
mutual_info	35	0.582*	0.587
mutual_info	40	0.569	0.591
rfe	10	0.575	0.591
rfe	20	0.574	0.590
rfe	25	0.570	0.588
rfe	30	0.581	0.586
rfe	35	0.566 [†]	0.585
rfe	40	0.566 [†]	0.587
rf_importance	10	0.591	0.585
rf_importance	20	0.598*	0.587
rf_importance	25	0.580	0.588
rf_importance	30	0.570	0.581
rf_importance	35	0.580	0.585
pca	–	0.511 [†]	0.561

DT is the only model for which feature selection produces a statistically significant improvement on Java. The best configuration (rf_importance, $k=20$) achieves 0.598, a gain of +0.026 ($p=0.001$). This is the **single statistically significant accuracy improvement across all models and methods** in this study.

On the multi-language dataset, DT benefits modestly but consistently from small k : mutual_info at $k=10$ achieves 0.602 (+0.019 over baseline). The pattern across

both datasets is the same: rf_importance and RFE are the reliable methods for DT, while chi2 and mutual_info with small k tend to hurt performance on Java by retaining individually weak but statistically dependent features that introduce noise into splits.

PCA is especially damaging for DT (-0.061 on Java, -0.022 on multi-language), as axis-aligned splits on principal components, which carry no direct semantic meaning, are particularly ineffective.

5.4. Logistic Regression

Table 9. LR accuracy by method and k . Java baseline: 0.390 (AUC: 0.651). Multi-language baseline: 0.395 (AUC: 0.758). † = significantly worse ($p<0.05$, Wilcoxon, Java only).

Method	k	Java		Multi-lang	
		Acc	AUC	Acc	AUC
chi2	10	0.345 [†]	0.669	0.372	0.733
chi2	20	0.355 [†]	0.674	0.383	0.744
chi2	25	0.359 [†]	0.650	0.401	0.752
chi2	30	0.372 [†]	0.653	0.396	0.754
chi2	35	0.375 [†]	0.658	0.399	0.755
chi2	40	0.390	0.655	0.393	0.756
mutual_info	10	0.322 [†]	0.623	0.376	0.730
mutual_info	20	0.350 [†]	0.636	0.393	0.757
mutual_info	25	0.354 [†]	0.634	0.388	0.756
mutual_info	30	0.382	0.648	0.389	0.755
mutual_info	35	0.391	0.658	0.387	0.756
mutual_info	40	0.385	0.654	0.388	0.758
rfe	10	0.368 [†]	0.678	0.377	0.738
rfe	20	0.386	0.664	0.372	0.742
rfe	25	0.388	0.651	0.386	0.750
rfe	30	0.386	0.649	0.377	0.753
rfe	35	0.398	0.652	0.383	0.750
rfe	40	0.387	0.651	0.384	0.751
rf_importance	10	0.376	0.671	0.351	0.676
rf_importance	20	0.400	0.676	0.372	0.743
rf_importance	25	0.388	0.658	0.382	0.750
rf_importance	30	0.402	0.654	0.373	0.747
rf_importance	35	0.399	0.661	0.383	0.751
rf_importance	40	0.396	0.656	0.385	0.750
pca	–	0.386	0.660	0.396	0.747

LR exhibits the most nuanced behaviour, with a notable dissociation between accuracy and AUC on both datasets. No configuration achieves a statistically significant accuracy improvement on Java. However, rf_importance at $k=20$ achieves the best accuracy (0.400, +0.010, $p=0.38$) alongside AUC of 0.676 (+0.025), while rfe at $k=10$ achieves the best AUC (0.678) at the cost of lower accuracy (0.368).

The interpretation is consistent with the VIF analysis: reducing to 20 features via rf_importance eliminates most features with $VIF > 10$, allowing LBFGS to find a more sta-

ble solution. The AUC improvement confirms better class discrimination, but the 74% OURS dominance constrains accuracy independently. On the multi-language dataset, `chi2` at $k=25$ achieves the best accuracy (0.401), marginally above baseline. The pattern is consistent: AUC responds to feature quality, accuracy does not.

Across both datasets, `chi2` and `mutual.info` with small k significantly degrade LR accuracy on Java, while `rf_importance` never does. The identity of selected features matters more than their count for LR.

5.5. XGBoost

Table 10. XGB accuracy by method and k . Java baseline: 0.632. Multi-language baseline: 0.709. † = significantly worse ($p < 0.05$, Wilcoxon, Java only).

Method	k	Java Acc	ML Acc
<code>chi2</code>	10	0.577 [†]	0.683
<code>chi2</code>	20	0.598 [†]	0.687
<code>chi2</code>	25	0.608 [†]	0.691
<code>chi2</code>	30	0.619 [†]	0.695
<code>chi2</code>	35	0.621 [†]	0.700
<code>chi2</code>	40	0.635	0.709
<code>mutual.info</code>	10	0.606 [†]	0.683
<code>mutual.info</code>	20	0.611 [†]	0.706
<code>mutual.info</code>	25	0.616 [†]	0.706
<code>mutual.info</code>	30	0.621 [†]	0.708
<code>mutual.info</code>	35	0.626 [†]	0.708
<code>mutual.info</code>	40	0.633	0.709
<code>rfe</code>	10	0.616 [†]	0.688
<code>rfe</code>	20	0.630	0.703
<code>rfe</code>	25	0.623	0.707
<code>rfe</code>	30	0.629	0.709
<code>rfe</code>	35	0.628	0.710
<code>rfe</code>	40	0.635	0.709
<code>rf_importance</code>	10	0.619 [†]	0.678
<code>rf_importance</code>	20	0.633	0.704
<code>rf_importance</code>	25	0.628	0.704
<code>rf_importance</code>	30	0.630	0.707
<code>rf_importance</code>	35	0.630	0.709
<code>rf_importance</code>	40	0.631	0.709
<code>pca</code>	–	0.617	0.656

XGB mirrors RF in its robustness: no configuration significantly improves accuracy, and aggressive selection hurts. The recovery curve is slightly slower than RF on Java (most methods require $k=40$ to match baseline), which aligns with XGB’s sequential boosting mechanism being more sensitive to removing features that correct residuals in early trees. On the multi-language dataset, `rfe` at $k=35$ achieves 0.710 (+0.001), effectively matching the baseline.

Notably, XGB with built-in L1 regularization (`reg.alpha=0.5`) already suppresses redundant

features during training, making external selection largely redundant — analogous to RF’s robustness via random subsampling. The baseline gap between RF (0.731) and XGB (0.709) on the multi-language dataset collapses entirely at $k=35$ –40, suggesting XGB benefits marginally more from having the redundant features stripped by an external method.

PCA degrades XGB substantially on both datasets (−0.015 Java, −0.053 multi-language).

5.6. Cross-Model Summary and the Redundancy Finding

Table 11 consolidates the best configuration per model.

Table 11. Best configuration per model on each dataset vs. respective baseline. Sig. = Wilcoxon significance (Java only, $n=16$).

Model	Best Config	Java		ML
		Acc	Sig.	Acc
RF	<code>chi2</code> , $k=40$	0.649	no	0.734
XGB	<code>rfe</code> , $k=35$	0.645	no	0.710
DT	<code>rf_imp</code> , $k=20$	0.598	yes	0.602
LR	<code>rf_imp</code> , $k=20$	0.400	no	0.401

The primary conclusion is that only DT benefits significantly from feature selection, and this holds across both datasets. However, framing the results purely around the accuracy ceiling obscures a practically important finding: **the 43-feature space can be reduced to approximately 20–25 features with no statistically significant accuracy loss for any model.** For RF and XGB, `rf_importance` at $k=20$ on Java achieves 0.642 and 0.633 respectively, gaps of only −0.005 and −0.001 from baselines of 0.647 and 0.632, both non-significant. On the multi-language dataset, `mutual.info` at $k=20$ exactly matches the RF baseline at 0.731.

This 50% dimensionality reduction without accuracy loss is independently meaningful: it identifies which features actually carry discriminative signal, reduces the metadata extraction cost for deployment, and provides a more interpretable model. The stable core across methods consists of structural features (chunk size, lines of code, changed files, cyclomatic complexity) with keyword features appearing only when the multi-language corpus provides enough cross-language variation for them to discriminate.

6. Reduced Feature Space: Manual Keyword Removal

The correlation and VIF analyses in Section 4 identified the 12 commit-message keyword features as forming a near-perfectly correlated cluster (pairwise $|r|$ ranging from 0.85

to 0.99, VIF values from 13 to 210). The automated feature selection methods handle this cluster by implicitly selecting one representative or distributing weight across the cluster, but they do so inconsistently across folds. A principled alternative is to remove 11 of the 12 keywords manually, retaining only `keywords_feature` as the single representative of the commit-message semantic signal. This reduces the feature space from 43 to 32 features.

The 11 removed features are: `keywords_fix`,
`keywords_bug`, `keywords_improve`,
`keywords_document`, `keywords_refactor`,
`keywords_update`, `keywords_add`,
`keywords_remove`, `keywords_use`,
`keywords_delete`, `keywords_change`.

Motivation. This experiment differs from automated selection in two ways. First, the removal is theory-driven rather than data-driven: we remove features because we know they are structurally redundant, not because a particular method ranks them low on a given fold. Second, it directly tests whether the keyword cluster, as a whole, contributes any discriminative signal beyond a single representative. If the 32-feature model matches or approaches the 43-feature baseline, this confirms that the entire keyword block is informationally equivalent to a single feature in this setting.

Results. The reduced 32-feature configuration produced performance comparable to that obtained using the complete feature set. Random Forest and XGBoost remained the best-performing models, achieving mean accuracies of 0.6462 and 0.6312, respectively. Decision Trees achieved 0.5687 accuracy, while Logistic Regression obtained 0.4137.

These results indicate that removing the selected keyword-related features does not substantially degrade predictive performance. The ranking of classifiers remained unchanged and the overall behaviour of the models was consistent with that observed using the full feature set.

Discussion. The findings suggest that the removed features contain information that is either redundant with other feature groups or contributes only marginally to classification performance. Consequently, a more compact feature set can be used without sacrificing predictive effectiveness.

Beyond maintaining predictive performance, the reduced feature set decreases feature extraction costs and improves model interpretability, making it more practical for real-world deployment.

7. Per-Language Analysis

Having established that RF is the best-performing model and that feature selection produces consistent patterns across both aggregate datasets, we now examine whether these patterns hold at the individual language level in the multi-language setting. For each of the 9 non-Java languages, we run RF with all four selection methods and the same $k \in \{10, 20, 25, 30, 35, 40\}$ range under the same cross-project 5-fold CV protocol. Since the Wilcoxon test lacks power with only $n=5$ fold observations (as discussed in Section 3.2), all results are reported descriptively.

7.1. Accuracy by Language

Table 12 reports the RF baseline accuracy per language together with the best result achieved under any feature selection configuration.

Table 12. RF cross-project accuracy per language: baseline (43 features) vs. best feature selection result. Δ = best minus baseline. All Wilcoxon tests yield $p=1.0$ due to $n=5$ fold observations; results are descriptive.

Language	Baseline	Best	Δ	Best Config
C	0.798	0.803	+0.005	rfe, $k=40$
C#	0.654	0.656	+0.002	rf_imp, $k=35$
C++	0.677	0.678	+0.001	chi2/rfe, $k=40$
Go	0.596	0.596	+0.000	chi2, $k=30$
JavaScript	0.574	0.578	+0.004	chi2, $k=30$
PHP	0.653	0.670	+0.017	chi2, $k=20$
Python	0.639	0.646	+0.006	mutual_info, $k=40$
Rust	0.732	0.732	+0.000	chi2, $k=35$
TypeScript	0.652	0.656	+0.004	rfe, $k=25$
Mean	0.664	0.668	+0.004	

Accuracy differences are negligible across all languages: the largest improvement is PHP (+0.017) and most languages change by less than 0.005. No language shows a consistent degradation from feature selection when $k \geq 20$. The aggregate picture from the multi-language analysis (Section 5.2) is therefore not the artifact of any single language dominating the mean.

7.2. Accuracy Recovery at $k=20$

A key question is whether the 50% dimensionality reduction at $k=20$ holds within individual languages or only in aggregate. Table 13 reports the accuracy gap at $k=20$ for `rf_importance`, the method with the most consistent stability.

At $k=20$, the mean gap across languages is only -0.007 . For five languages (C, C++, Go, TypeScript, and marginally PHP) the gap is within ± 0.007 . The languages with the largest gaps at $k=20$ are JavaScript (-0.018) and C#

Table 13. RF accuracy gap at $k=20$ (rf_importance) relative to baseline, per language.

Language	Baseline	$k=20$	Δ
C	0.798	0.795	-0.004
C#	0.654	0.642	-0.013
C++	0.677	0.677	-0.000
Go	0.596	0.594	-0.002
JavaScript	0.574	0.556	-0.018
PHP	0.653	0.646	-0.007
Python	0.639	0.629	-0.010
Rust	0.732	0.722	-0.010
TypeScript	0.652	0.651	-0.001
Mean	0.664	0.657	-0.007

(-0.013), both of which recover to within 0.002 of their respective baselines by $k=25-30$. This suggests that for some language ecosystems the optimal compact subset requires slightly more features than 20, but the redundancy finding is robust: between 20 and 30 features suffice across all languages.

7.3. Redundancy Structure by Language

The number of highly correlated feature pairs ($|r| \geq 0.85$) varies substantially across languages, as shown in Table 14.

Table 14. Feature redundancy and PCA compactness per language. Corr. pairs: number of feature pairs with $|r| \geq 0.85$. PCA-95%: components needed to retain 95% of variance.

Language	Corr. pairs	PCA-95%
C	87	15
Rust	99	17
C++	70	16
Go	39	19
C#	31	19
JavaScript	14	24
TypeScript	16	23
PHP	7	26
Python	11	23

There is a clear pattern: systems-level languages (C, Rust, C++) exhibit far more redundancy than scripting/web languages (PHP, Python, JavaScript, TypeScript). C and Rust have 87 and 99 highly correlated pairs respectively, driven by the keyword cluster being almost perfectly collinear in these ecosystems: in large-scale C and Rust projects, commit vocabulary is highly uniform (all keywords move together), making the entire block informationally equivalent to a single feature. PHP and Python projects show much more heterogeneous commit vocabularies, with only 7 and 11 highly correlated pairs. The PCA compactness figures confirm this: C requires only 15 components for 95% variance, while PHP requires 26.

Crucially, this redundancy difference does not translate into a corresponding accuracy difference. Despite C having 87 correlated pairs, RF accuracy (0.798) is among the highest. This reinforces the conclusion that the redundancy is absorbed by RF’s internal subsampling and does not harm performance, but also that removing it does not help.

7.4. Stable Features Across Languages

Despite the heterogeneity in redundancy structure, the stable feature subsets (selected in $\geq 80\%$ of folds by rf_importance) share a consistent core across all 9 languages: chunk size features (chunkRelSize, chunkAbsSize, chunkAbsSizeOURS), lines-of-code features (locAddedOURS, locAddedTHEIRS, locTentativeFile), changed files counts (changedFilesOURS, changedFilesTHEIRS), commit counts (commitCountOURS, commitCountTHEIRS), chunkPositionQuarter, and differentDevelopers.

These structural features appear in the stable subset of every language. Keyword features, by contrast, appear in stable subsets for some languages (notably C, Rust, and Go under certain methods) but not others (PHP, Python), confirming the language-dependent discriminative value of commit vocabulary identified in the aggregate analysis. The cyclomaticComplexity family appears in most languages but with lower consistency, suggesting contextual complexity contributes modestly and unevenly across ecosystems.

8. Discussion

8.1. Feature Redundancy Is Severe But Gracefully Absorbed

The most consistent finding across all experiments is that 58% of the 43 features (25 features with VIF > 10) are severely multicollinear, yet this redundancy has a paradoxical effect: it neither hurts RF and XGB (which absorb it through internal mechanisms) nor helps when removed (because the retained features already capture all available discriminative signal).

This distinguishes the problem from settings where redundancy actively harms performance. In those settings, feature selection improves accuracy by removing correlated noise. Here, the redundancy is *uninformative redundancy*: the highly correlated features all encode the same signal, and that signal is partially useful. Removing the redundant copies changes the encoding but not the information content, leaving accuracy unchanged.

The practical consequence is that 20–25 features encode effectively all the discriminative information in the 43-

feature space. This means future work on metadata-based approaches can safely use a compact representation, reducing data collection cost and improving interpretability without sacrificing predictive power.

8.2. Which Features Carry Discriminative Signal?

The stable feature analysis across all experiments points to a clear hierarchy. Structural features are universally informative: chunk size, lines of code, changed file counts, and chunk position appear in stable subsets across all models, all languages, and both datasets. These features directly characterize the scope and nature of a conflict and are the primary drivers of resolution pattern prediction.

Authorship and collaboration features (commit counts, developer counts, differentDevelopers) form a secondary tier: consistently stable in most settings but with some language-level variation. Temporal features (mergeDurationDays, contributionDurationDays) appear in stable subsets on the multi-language dataset but less so on Java, possibly because temporal patterns vary more across language ecosystems than within a single-language corpus.

Keyword features occupy the lowest tier. Within the Java dataset, they are consistently excluded from stable subsets; keyword correlations within a monolingual corpus are so high that the features offer no additional signal beyond commit count. In the multi-language dataset and in individual languages with heterogeneous commit cultures (PHP, Python), a single keyword feature enters stable subsets and contributes marginally. The manual removal experiment (Section 6) further demonstrated that a substantial portion of the keyword-based features can be removed without materially affecting predictive performance. This finding suggests that the proposed feature space contains redundancy and that compact feature subsets can provide a favourable trade-off between accuracy, computational cost, and interpretability.

8.3. Model Sensitivity and the DT Exception

The divergence in feature selection sensitivity between models is explained by their internal architectures. RF and XGB both have mechanisms that implicitly handle redundancy: random feature subsampling at each split (RF) and L1 regularization on tree weights (XGB). These mechanisms make external selection redundant at large k and harmful at small k (where informative features may be discarded). DT lacks both, making it genuinely sensitive to the quality of the input feature set. This explains why `rf_importance` at $k=20$ is the only configuration that produces a statistically significant improvement.

LR’s behaviour is distinct: it is not simply sensitive to redundancy, it is *broken* by it. With $VIF = \infty$ for 9 features,

the design matrix is rank-deficient and the LBFGS optimizer cannot find a stable solution. Feature selection partially repairs this: `rf_importance` at $k=20$ eliminates most of the infinite-VIF features and produces a measurable AUC improvement (+0.025), though accuracy remains constrained by class imbalance regardless of feature quality.

8.4. The Performance Ceiling Persists

Despite principled feature engineering across two datasets and six values of k , the performance ceiling identified by (Authors, 2026) remains essentially unchanged. RF accuracy on Java is 0.647 at baseline and 0.649 at best; on the multi-language dataset it is 0.731 at baseline and 0.734 at best. The improvements for DT (+2.6% on Java, +1.9% on multi-language) and LR (AUC +2.5%) are real but modest.

This confirms that the ceiling is a property of the feature representation, not of the model or the feature selection method. Metadata features capture *contextual* signals about conflicts (who changed what, how large, how long the branch ran) but not *semantic* signals about the code itself. Two conflicting chunks with identical metadata can require completely different resolutions depending on the actual code content. Overcoming this ceiling likely requires models that operate on the code directly, such as large language models that can approximate developer intent at the semantic level.

8.5. Limitations

Several limitations apply. First, RFE uses a DT as the internal estimator for all models due to computational constraints, which may introduce importance ranking bias for RF and XGB. Second, LR convergence is impeded by the rank-deficient feature matrix; a numerically more stable solver or explicit ridge regularization might yield different AUC results. Third, hyperparameters are fixed without tuning per feature subset, which may disadvantage some configurations. Fourth, the manual keyword removal experiment (Section 6) results are pending and may modify the conclusions about keyword informativeness. Fifth, the per-language analysis lacks statistical testing power ($n=5$ folds), so language-level differences are descriptive.

9. Conclusion

We conducted a systematic study of feature engineering and selection for classification-based merge conflict resolution on a Java-only intra-project dataset and a large-scale multi-language cross-project dataset, extending the evaluation to $k \in \{10, 20, 25, 30, 35, 40\}$ feature subsets and complementing the aggregate analysis with a per-language examination of RF, the consistently best-performing model.

The central empirical finding has two faces. On the accuracy side, feature selection provides a statistically sig-

nificant improvement only for Decision Tree (+2.6% on Java, $p=0.001$, `rf_importance`, $k=20$). RF and XGB are robust to redundancy by design, and no selection method improves their performance significantly. For LR, selection improves AUC (+2.5%) without achieving significant accuracy gains. PCA degrades all models.

On the redundancy side, the 43-feature space can be reduced to approximately 20–25 features with no statistically significant accuracy loss for any model on either dataset. This finding holds across all 9 individual languages, with optimal compact subsets requiring between 20 and 30 features depending on the ecosystem. The stable core comprises structural features (chunk size, lines of code, changed files, cyclomatic complexity) which appear universally, while keyword features are informative only in cross-language settings and dispensable within a monolingual corpus. These conclusions are reinforced by the manual removal experiment (Section 6), which provides a theory-driven test of the keyword cluster’s dispensability.

The per-language analysis reveals that redundancy is substantially higher in systems-level languages (C: 87 correlated pairs, Rust: 99) than in scripting/web languages (PHP: 7, Python: 11), driven by the near-perfectly collinear keyword cluster in ecosystems with uniform commit vocabularies. Despite this variation, the stable feature core is consistent across all languages, and the accuracy recovery curve at $k=20$ –25 holds regardless of ecosystem.

Taken together, these results reinforce and extend the conclusion of (Authors, 2026): the performance ceiling of classical ML (~ 0.65 – 0.73) is not a consequence of suboptimal feature selection or model choice. Even with principled feature engineering across two datasets, six feature subset sizes, and individual language analysis, the ceiling persists. The discriminative limit of metadata-level features has been mapped, and meaningful further progress likely requires models that operate on the actual code content of conflicting chunks, such as large language models capable of approximating developer intent at the semantic level.

References

- Authors, A. A dead end for classical machine learning in merge conflict resolution?, 2026. Manuscript under review.
- Boll, A., Dok, Y. V., Ohrndorf, M., Schultheiß, A., and Kehr, T. Towards semi-automated merge conflict resolution: Is it easier than we expected? In *Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering (EASE’24)*, pp. 282–292. ACM, June 2024.
- Breiman, L. Random forests. *Machine Learning*, 45(1): 5–32, 2001.
- Chen, T. and Guestrin, C. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD’16)*, pp. 785–794. ACM, 2016.
- Elias, P., Campos, H. D. S., Ogasawara, E., and Murta, L. G. P. Towards accurate recommendations of merge conflicts resolution strategies. *Information and Software Technology*, 164:107332, December 2023.
- Ghiotto, G., Murta, L., Barros, M., and van der Hoek, A. On the nature of merge conflicts: A study of 2,731 open source Java projects hosted by GitHub. *IEEE Transactions on Software Engineering*, 46(8):892–915, 2020.
- Guyon, I. and Elisseeff, A. An introduction to variable and feature selection. *Journal of Machine Learning Research*, 3:1157–1182, 2003.